An Adaptive LLM-Based Pipeline for Semantic Table Annotation

Contents

Figures

Tables

1. Introduction

Semantic Table Annotation is an important task for transforming raw tabular data into semantically meaningful and machine-interpretable information. Tables are widely used in databases, open data portals, spreadsheets, data lakes, and web sources, but their structure alone is often not enough to reveal the actual meaning of their contents. Column names may be missing, ambiguous, or overly generic, while cell values may be abbreviated, noisy, coded or dependent on the surrounding context. As a result, interpreting a table automatically requires more than reading its surface structure; it requires linking table elements to meaningful entities, concepts, or ontology classes.

This project focuses on implementing a semantic table annotation pipeline inspired by the paper *An LLM Agent-Based Complex Semantic Table Annotation Approach*. The selected paper proposes an LLM-agent-based method for Semantic Table Annotation, with emphasis on Column Type Annotation (CTA) and Cell Entity Annotation (CEA). Its approach combines LLM-based reasoning, knowledge graph lookup, candidate ranking, context-supported selection, and workflow adaptation according to the characteristics of each table. These ideas are particularly relevant for handling complex annotation cases such as weak column names, weak cell values, spelling errors, abbreviations and homonyms.[1]

1.1 Project Objective and Scope

The main objective of this project is to design and implement a modular Semantic Table Annotation pipeline for CTA and CEA tasks. The implementation follows the core workflow ideas of the selected paper, while adapting them into a clean, testable, and extensible Python project structure.

The system follows a paper-inspired agentic design, but it does not implement a fully autonomous ReAct loop at runtime. Instead, it uses explicit workflow routing based on the characteristics of each table and column. More specifically, the routing mechanism examines whether column headers and cell values are semantically strong or weak and then selects the most suitable annotation strategy. The implementation mainly focuses on two Semantic Table Annotation subtasks:

- CEA: linking cell values to knowledge graph entities.
- CTA: assigning a semantic type or ontology class to a column.

The project does not aim to reproduce the full official benchmark evaluation of the original paper with complete precision, recall, and F1-score measurements over all Tough Tables and BiodivTab

[1] Liu et al., "From Tabular Data to Knowledge Graphs."

datasets. Instead, the implemented system is evaluated through custom examples, selected benchmark-style tables, generated reports, and qualitative comparison of the produced annotations.

1.2 Project Structure

The project is organized in a modular repository structure, consisting of mainly ***.py*** files. Each folder has a specific responsibility, and the main pipeline coordinates the execution of the individual modules. The workflow is the following:

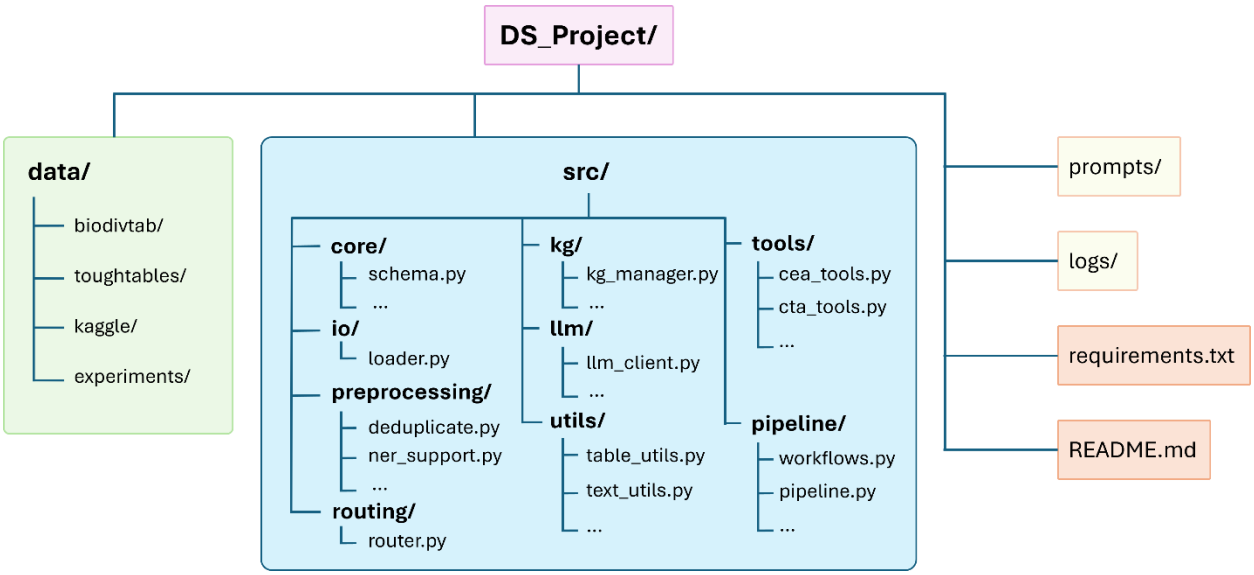

Figure 1-1: Project Structure

The ***src/core/*** folder contains the main data structures used to represent tables, columns, and cells. The ***src/io/*** folder is responsible for loading CSV files into the internal representation. The ***src/preprocessing/*** folder contains cleaning, representative value extraction, NER support, and value enrichment logic. The ***src/routing/*** folder contains the routing mechanism that assigns workflow scenarios. The ***src/kg/*** folder handles interaction with knowledge graph sources. The ***src/llm/*** folder manages LLM providers, prompts, and model calls. The ***src/tools/*** folder contains the main annotation tools for topic detection, CEA, and CTA. Finally, the ***src/pipeline/*** folder coordinates the full execution and generates the final report. Outside the source code, the ***prompts/*** folder stores prompt templates, the ***data/*** folder stores benchmark-style and custom datasets, the ***data/experiments/*** folder contains selected experiment files, and the ***logs/*** folder stores generated run reports. This structure keeps the implementation, data, prompts, and outputs separate, making the project easier to test, inspect, and extend.

2. Background and Related Work

Semantic Table Annotation, also referred to as Semantic Table Interpretation, is the process of extracting meaningful information from tabular data with respect to a semantic artifact, such as an ontology or a knowledge graph. Instead of treating a table only as rows and columns, semantic annotation attempts to identify what the table elements represent and how they can be connected to structured knowledge. This is useful in data integration, knowledge graph construction, dataset search, and data lake management, where heterogeneous tables need to be understood in a consistent way. The **SemTab** challenge also defines this research area around tabular data to knowledge graph matching, with tasks such as CEA, CTA, and CPA. [2] These tasks are important because a raw table may be understandable to a human reader but still difficult for a system to connect automatically to external knowledge.

2.1 Semantic Table Annotation

The selected paper focuses mainly on two Semantic Table Annotation tasks: Column Type Annotation and Cell Entity Annotation. Column Type Annotation (CTA) assigns a semantic type or ontology class to a column, while Cell Entity Annotation (CEA) links individual cell values to entities in a knowledge graph. A third related task is Column Property Annotation (CPA), which identifies semantic relationships between columns and belongs to the wider Semantic Table Annotation area.

Table 2-1: STA Example

Table element	Example input	Annotation task	Example output
Cell	Apple	CEA	DBpedia: Apple Inc.
Cell	United States	CEA	DBpedia: United States
Column	Apple, Microsoft, Amazon	CTA	Company / Organisation
Column	United States, China	CTA	Country
Column pair	company, country	CPA	headquarters country / origin country

2.2 Challenges in Complex Tables

Semantic annotation becomes difficult when tables contain weak or incomplete semantic evidence. The selected paper focuses on complex tables where the meaning of a column or cell is not directly available from the surface text. These cases are important because many real-

[2] "Semantic Web Challenge on Tabular Data to Knowledge Graph Matching."

world datasets are not clean semantic tables; they often come from spreadsheets, web tables, open data portals, or exported database files.

One common problem is semantic loss of column names. A column may be named *col0, col1, C*, or another generic label, which does not describe the meaning of the values. In this case, the system must infer the topic of the column from its cell values. Another problem is semantic loss of cell values. A column may have a meaningful header, but its cells may be numeric, empty, coded, or abbreviated, making entity linking unreliable.

Other challenges include strict ontology hierarchy, homonyms, spelling errors, and abbreviations. Strict hierarchy means that the predicted class should not be too broad or too narrow. For example, a column containing pet types should not simply be annotated as Animal if a more suitable class is available. Homonyms occur when the same name may refer to different entities, so the system must use row context to distinguish between possible meanings. Spelling errors and abbreviations can also prevent direct knowledge graph lookup, especially in domain-specific tables.[3]

These challenges motivate the use of both knowledge graphs and LLMs. Knowledge graphs provide structured candidate entities and classes, while LLMs can use table context to select between candidates, infer missing topics, and support fallback decisions when direct lookup is unreliable.

2.3 Paper's Approach and Architecture

The project is based on the paper *An **LLM Agent-Based Complex Semantic Table Annotation Approach***[4], which proposes a ReAct-based agent for CTA and CEA. The paper combines LLM reasoning with external tools, including preprocessing, column topic detection, knowledge graph enhancement, CTA candidate ranking, context-supported CEA selection, and context-supported CTA selection. The purpose of this architecture is to use knowledge graphs for structured candidate generation and LLMs for context-aware decision making.

The paper follows the **ReAct** idea, where language models combine reasoning with actions such as tool use or interaction with external information sources. This is relevant for semantic table annotation because the system should not rely only on generated answers, but should also consult external knowledge sources when selecting candidate entities and column types. ReAct was originally proposed as a way to interleave reasoning traces and task-specific actions, allowing models to interact with sources such as knowledge bases or external environments during problem solving. [5]

[3] Geng et al., "An LLM Agent-Based Complex Semantic Table Annotation Approach."
[4] Geng et al., "An LLM Agent-Based Complex Semantic Table Annotation Approach."
[5] Yao et al., "ReAct."

In the paper's architecture, knowledge graph lookup is used to retrieve candidate entities and ontology classes, while the LLM is used to select the most suitable result based on table context. DBpedia is suitable for this role because it extracts structured information from Wikipedia and makes it accessible as linked data. Wikidata can also support this role as a collaborative knowledge base that stores structured entities and relations.[6]

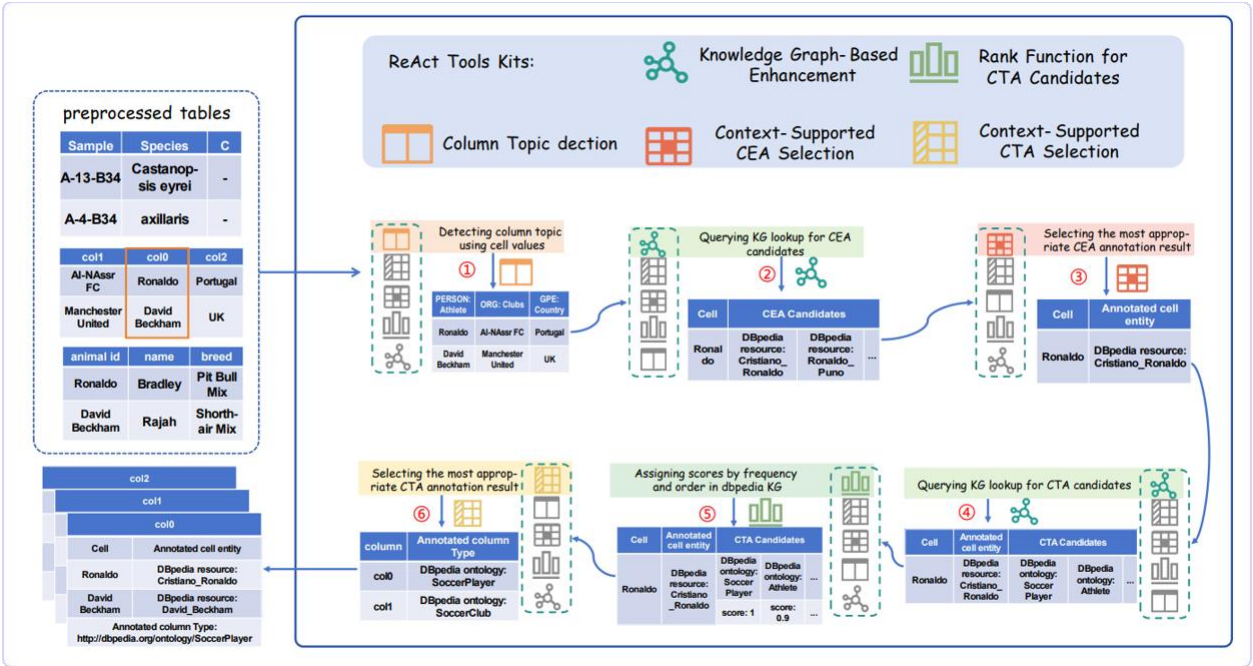

Figure 2-1: Paper's ReAct Framework Architecture

A central idea of the paper is that different tables require different annotation workflows. When column names are weak, but cells are meaningful, the system first infers a column topic and then performs entity and type annotation. When column names are meaningful but cell values are weak, CEA is skipped and CTA relies mainly on the column name and the surrounding table context. When both headers and cells are meaningful, topic detection is skipped and the remaining annotation tools are applied directly. In this project, the same general logic is implemented through an explicit routing component, which selects the appropriate workflow before executing the relevant pipeline tools.

[6] Bizer et al., "DBpedia - A Crystallization Point for the Web of Data."

3. System Architecture

The architecture was designed to remain close to the workflow logic of the selected paper, while keeping the implementation explicit and easy to inspect. At a high level, the system receives a table as input and produces an annotated report as output. The pipeline begins by loading the table and applying basic preprocessing. Then, representative values are extracted from each column to support later stages such as NER, value enrichment, topic detection, and routing. After that, the router classifies the table and its columns into workflow scenarios.

Instead of allowing the LLM to freely decide every next action during runtime, the system uses predefined pipeline stages and an explicit routing component. The router examines the semantic strength of column headers and cell values and then determines which workflow should be followed. This keeps the implementation close to the paper's adaptive logic, while making the execution easier to debug and explain.

Depending on the selected scenario, the system may execute topic detection, CEA candidate generation, CEA selection, and CTA selection. Knowledge graph lookup is used when candidate entities or ontology classes are needed, while the LLM is used for context-supported decisions. The final output is a compact TXT report that summarizes the execution, skipped stages, routing decisions, candidate information, and final annotations.

The architecture separates the project into four main concerns: table representation, pipeline processing, external semantic support, and reporting. Table representation defines how tables, columns, and cells are stored internally. Pipeline processing coordinates the annotation stages. External semantic support includes LLM calls, prompts, and knowledge graph lookup. Reporting records the execution trace and final results.

3.1 Pipeline

The complete pipeline is organized into the following main stages:

Table Loading: The input CSV file is loaded and converted into the internal table representation.

Preprocessing: Cell values are cleaned and normalized so that later stages operate on more consistent data.

Representative Value Extraction: A small set of representative values is selected for each column. These values are used to reduce unnecessary processing while still preserving useful column evidence.

NER and Value Enrichment: Optional enrichment steps are applied to support semantic understanding. NER provides hints about possible entity types, while value enrichment can help normalize or clarify representative values.

Routing: The router examines header strength and cell-value strength in order to decide which workflow scenario should be followed.

Topic Detection: When headers are weak but cells are meaningful, the system uses the LLM to infer a more meaningful working column name.

CEA Candidate Generation and Selection: For strong-cell workflows, the system retrieves entity candidates from a knowledge graph and uses the LLM to select the most suitable entity based on row and column context.

CTA Candidate Ranking and Selection: The system derives or infers column type annotations. In strong-cell cases, CTA candidates can be collected from annotated entities and ranked. In weak-cell cases, the LLM performs context-supported CTA using the column name and surrounding table context.

Reporting: The final stage generates a compact TXT report describing the execution and results.

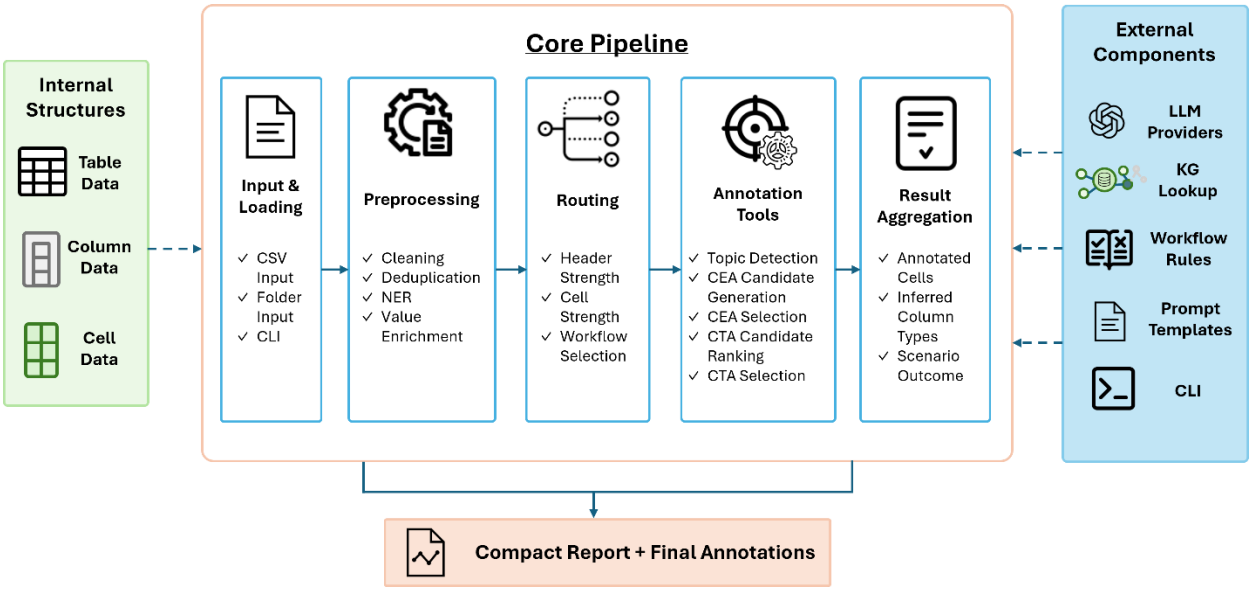

Figure 3-1: Core Implementation Pipeline

This staged design makes the system easier to control. Each stage receives the table object, updates it with new information, and passes it to the next stage. Some stages are always

executed, while others are skipped depending on the routing decision. This allows the system to adapt to different table situations without manually changing the pipeline for each experiment.

3.2 Core Data Model

The internal data model is based on three main concepts: table, column, and cell. The table object represents the whole input file and stores global information about the dataset and pipeline execution. Each column object stores information about its name, semantic status, routing scenario, and final column type. Each cell object stores its raw and cleaned value, along with any annotation information produced during the pipeline.

This structure allows information to be accumulated progressively. For example, preprocessing updates cell values, routing updates column-level workflow decisions, CEA updates cell-level entity annotations, and CTA updates column-level type annotations. As a result, the same internal object can carry the full annotation state from the beginning to the end of the pipeline.

A simplified view of the data model can be shown as:

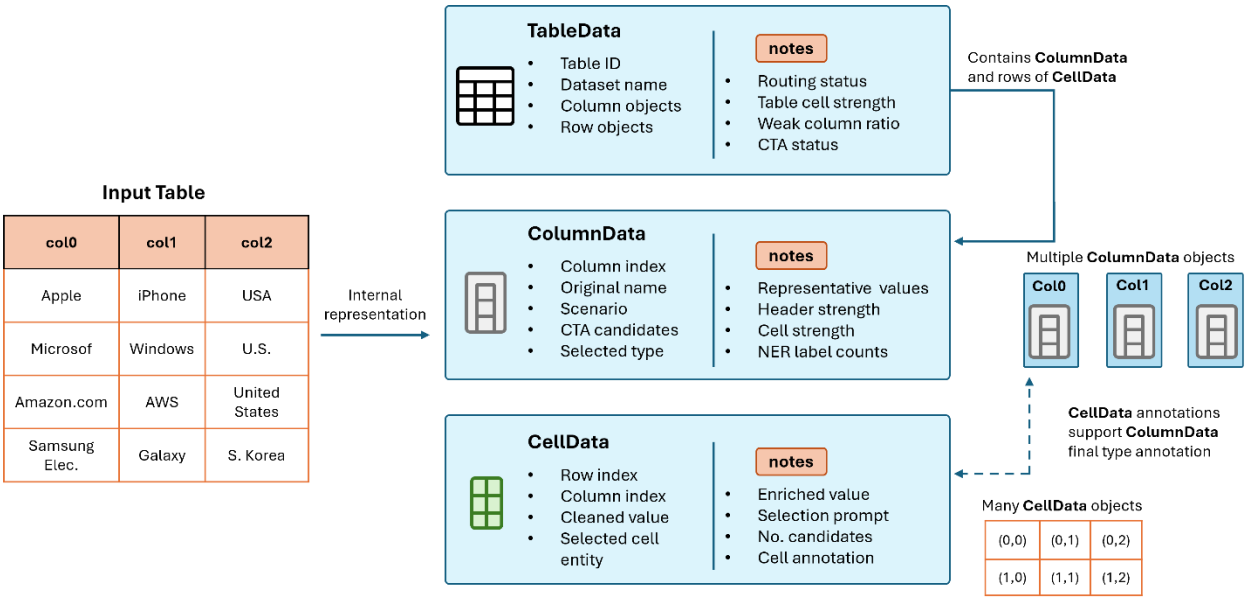

Figure 3-2: Core Data Model

3.3 External Components

The system also depends on external semantic components. These components are separated from the main pipeline so that they can be replaced or configured without changing the full architecture.

LLM providers are used for tasks that require contextual judgment, such as topic detection, entity selection, value enrichment, and column type selection. The system is designed to support multiple providers, including online APIs and local models.

Knowledge graph sources are used to retrieve structured candidates for CEA and CTA. In strong-cell workflows, cell values can be sent to a knowledge graph lookup component to retrieve possible entity matches. Annotated entities can then support the generation of column type candidates.

Prompts define how the system communicates with the LLM. Keeping prompts separate from the pipeline makes it easier to adjust the model instructions without rewriting the code.

Reports provide the final trace of the system execution. They are not only output files, but also an inspection mechanism. They help verify whether the correct workflow was selected, whether stages were skipped properly, and what final annotations were produced.

Input **Datasets** can be provided either as a single CSV file or as a folder of CSV files, depending on the experiment.

Finally, the **CLI** acts as the external execution interface of the system. It allows the user to run the pipeline on a single CSV file or a folder of CSV files, select the LLM provider and model, choose the KG source, control CEA/CTA limits, and generate separate reports for each experiment.

4. Implementation

This section describes the main implementation components of the system. The focus is not on individual functions, but on the role of each module inside the complete semantic annotation pipeline. The implementation follows the workflow ideas of the selected paper, while organizing them into a modular environment that can be tested, modified, and executed through the command line.

4.1 Preprocessing & Representative Values

The first implementation stage prepares the input table before any semantic annotation is performed. This part of the system is mainly implemented in the ***src/preprocessing/*** folder, with the table first loaded through ***src/io/loader.py*** and represented using the internal structures from ***src/core/***. The goal of this stage is to transform the raw CSV input into a cleaner and more consistent form that can be used by the later routing, CEA, and CTA components.

Preprocessing includes basic text cleaning, normalization of empty or missing values, and preparation of cell contents for later lookup or LLM-based reasoning. This step is important because real tables often contain noisy values, inconsistent spacing, missing entries, abbreviations, or values that are difficult to interpret directly. The selected paper also emphasizes preprocessing as a necessary support step for handling spelling errors and abbreviations before semantic annotation.

After basic preprocessing, the system extracts **representative values** for each column. Instead of sending every cell of a column to every later component, a smaller set of representative values is selected and stored as column-level evidence. These values are used by later stages such as NER support, value enrichment, routing, and topic detection. The paper uses a similar idea through deduplication and representative cell selection. Representative cells help reduce redundant processing and lower LLM usage, while still keeping values that reflect the semantics of the column.

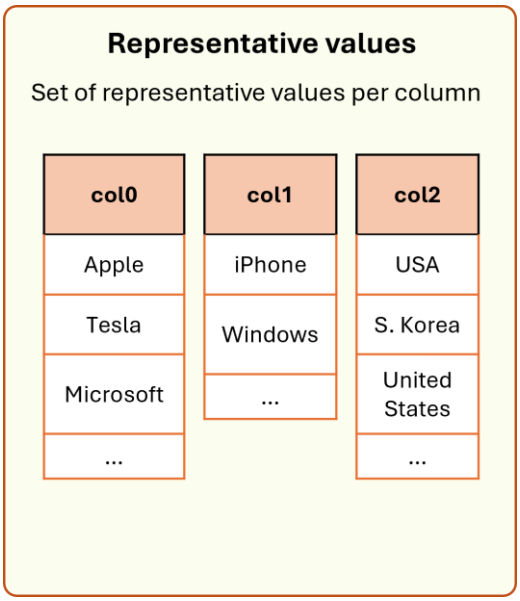

Figure 4-1: Representative Values example

After representative values are selected, the pipeline can apply two optional support steps: **NER** support and value enrichment. NER support (***src/preprocessing/ner_support.py***) is used to detect possible entity-type patterns inside the representative values of each column, such as person names, organizations, locations, dates, or other recognizable entity categories. These hints do not produce the final annotation, but they provide additional evidence that can support later semantic decisions.

Value enrichment (***src/preprocessing/value_enrichment.py***) is used as an optional LLM-based normalization step. Its purpose is to clarify or clean representative values before they are used in later stages, especially when values contain abbreviations, noisy formatting, or unclear textual forms. In the implemented system, value enrichment is applied only to representative values rather than every cell, in order to keep the number of LLM calls manageable. The output is stored as an enrichment map, so later components can use the clarified value when useful while still preserving the original cell value.

Together, preprocessing, representative values, NER support, and value enrichment form the semantic preparation layer of the pipeline. They do not replace knowledge graph lookup or LLM-based annotation, but they improve the quality of the evidence passed to routing, topic detection, CEA, and CTA.

4.2 Routing

The routing module is one of the central components of the implementation, because it controls which annotation workflow should be followed for each table or column. It is implemented in ***src/routing/router.py*** and acts as the connection between the general pipeline and the paper-inspired workflow scenarios. Its purpose is to decide which annotation tools are relevant for the current table. Instead of allowing a fully autonomous agent to decide tool usage at runtime, the implementation makes this decision explicitly through routing rules.

The **router** examines two main sources of evidence: the column header and the column values. Header strength is used to determine whether the column name is meaningful or weak. For example, names such as ***col0, col1***, or very short/generated headers are treated as weak because they do not provide enough semantic information. Cell strength is estimated from the representative values of the column. Values that are textual and entity-like provide stronger semantic evidence, while values that are mostly numeric, date-like, identifier-like, empty, or coded are treated as weaker evidence for routing.

This routing logic is based on the paper's idea that different table situations require different annotation strategies. The paper distinguishes between cases where column names are weak but cells are meaningful, cases where column names are meaningful but cells are weak, and cases where both headers and cells are meaningful. Each case requires a different combination of tools, such as topic detection, knowledge graph lookup, **CEA** selection, or context-supported **CTA** selection. The router also computes table-level information, such as the ratio of weak-cell columns and the distribution of workflow scenarios.

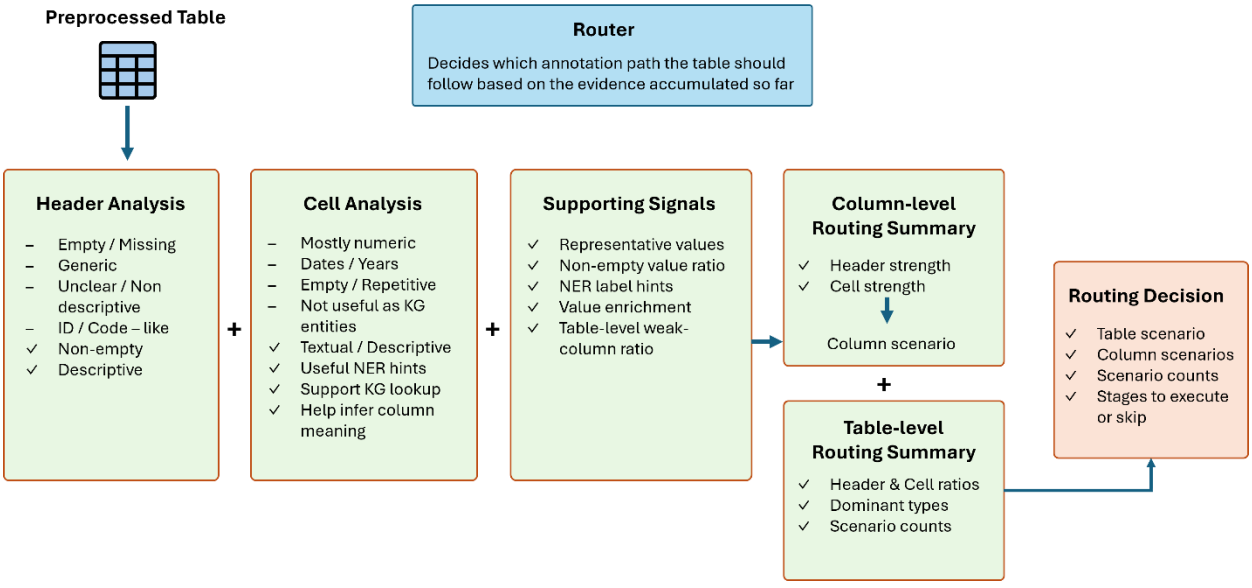

Figure 4-2: Router Module

4.3 Topic Detection

Topic detection is implemented as a support tool for cases where the original column header does not provide enough semantic information. It is mainly handled in ***src/tools/topic_detection.py***. The purpose of this module is to infer a more meaningful working name for a column by using the values contained in that column as evidence.

This component follows the idea of the paper, where column topic detection is used to address semantic loss of column names. When a header is generic or automatically generated, such as ***col0*** or ***column_1***, the system cannot rely on the header alone. In that case, the **LLM** is asked to inspect representative cell values and infer a compact topic that better describes the column.

In the implemented pipeline, topic detection is not applied to every column. It is executed only when the routing stage indicates that the header is weak and the cell values provide enough semantic evidence. If the header is already meaningful, or if the table belongs to a weak-cell scenario, this step is skipped. This avoids unnecessary LLM calls and prevents the system from inferring topics from unreliable cell values.

The output of topic detection is stored as an inferred column name. This name can then be used by later stages, especially CEA and CTA, as additional semantic context. Its role is to improve the context available to the annotation tools. It is used only when it adds useful semantic information, while the final annotation decision remains part of the later CTA process.

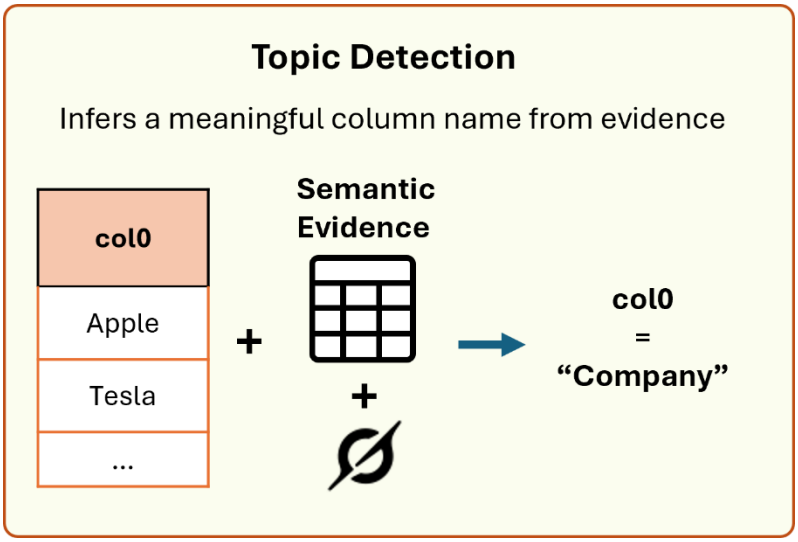

Figure 4-3: Topic Detection module

4.4 CEA Candidate Generation & Selection

Cell Entity Annotation is responsible for linking cell values to entities in a knowledge graph. In the implementation, this logic is mainly handled in ***src/tools/cea_tools.py***, with candidate lookup supported by the modules in ***src/kg/***. The goal of this component is to take a cell value, retrieve possible entity matches, and then select the most appropriate entity using the surrounding table context.

The process is divided into two main parts. First, the **candidate generation** step queries the selected knowledge graph source for possible entities matching the cell value. This produces a small candidate set instead of directly assigning an annotation. The number of returned candidates is controlled by the CLI parameter for KG candidate limit. Then, the **candidate selection** step uses the **LLM** to choose the most suitable entity from this set. The LLM is not asked to generate an entity freely, but to decide between structured candidates returned by the knowledge graph.

The selection prompt includes the cell value, the column name or inferred column name, the other values in the same row, and the candidate entities. The surrounding row and column context helps resolve ambiguous values and homonyms.

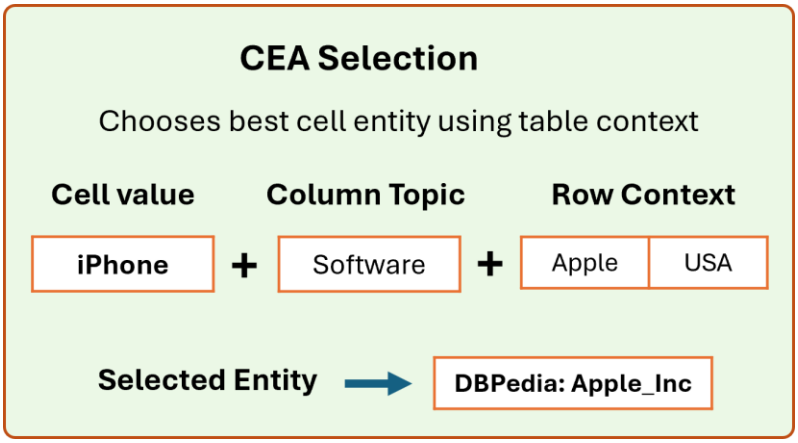

Figure 4-4: CEA Selection

The implementation also includes a reuse mechanism based on string similarity. If a cell is very similar to a previously annotated cell, the system can reuse the existing annotation instead of repeating the full LLM selection process. It makes use of **Levenshtein distance** to reduce redundant cell annotations and lower time and token costs.

The CEA module acts as the bridge between raw cell values and knowledge graph entities as it provides important entity-level evidence that can later support Column Type Annotation.

4.5 CTA Candidate Ranking & Selection

Column Type Annotation is responsible for assigning a semantic type or ontology class to each column. In the implementation, this component is mainly handled in ***src/tools/cta_tools.py***, with support from the knowledge graph modules, the LLM layer, and, when available, the results produced by **CEA**.

The **CTA** process can use two kinds of evidence. When entity annotations are available from **CEA**, the system can retrieve ontology classes or types associated with the selected entities. These become candidate column types. Since multiple entities may return multiple possible classes, the system ranks the candidates before the final selection step. The ranking is based on both frequency and order of occurrence in the candidate lists.

After candidate ranking, the LLM is used to select the most suitable column type from the filtered candidate set. The LLM receives the column name or inferred working name, representative column values, available cell annotations, and the ranked type candidates. This context-supported selection is important because the most frequent candidate is not always the best one. The LLM can use the surrounding context to avoid types that are too broad, too narrow, or semantically inappropriate for the column.

When knowledge graph candidates are not available or not useful, the CTA module can still produce a context-supported type using the LLM. In that case, the decision relies more on the column name, representative values, and surrounding table context. This makes CTA the final semantic interpretation step of the pipeline, because every column should receive a final type label even when entity-level evidence is limited. The output of CTA is stored as the final column type annotation.

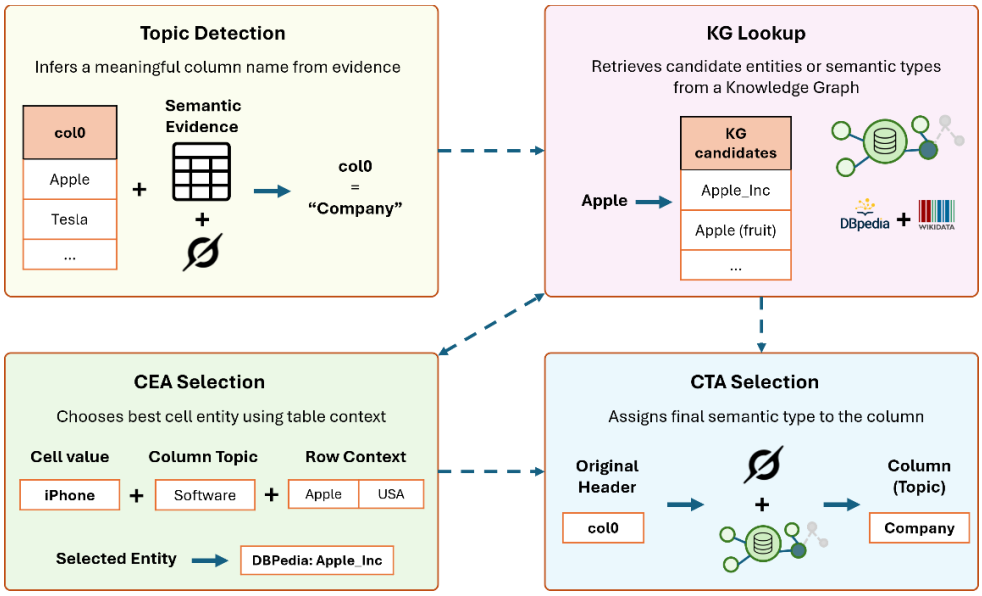

Figure 4-5: Entire STA Toolkit

The final CTA result is stored at column level. Depending on the available evidence, the source of the type may be knowledge-graph-supported or LLM-supported.

4.6 Reporting & Logging

Reporting and logging are important parts of the implementation because they make the annotation process inspectable. The system does not only produce final CTA and CEA outputs, but also generates a structured .txt report that shows how the pipeline reached those outputs. This is useful for debugging, experimentation, and understanding whether each stage behaved as expected. The report is generated automatically during pipeline execution and is stored in the ***src/logs/runs*** folder.

The reporting logic is mainly implemented in ***src/pipeline/reporting.py***, with reusable formatting support from ***src/pipeline/reporting_helpers.py*** and general logging utilities from ***src/utils/log_utils.py***. The report is generated automatically during pipeline execution and is stored in the run logs folder. Each run creates its own report, so different experiments can be compared without overwriting previous results.

The report includes compact summaries of the most important intermediate results, such as representative values, routing decisions, topic detection results, CEA candidates, selected cell entities, and final CTA labels. It is also used as the main inspection artifact in the experiments. Since the evaluation is qualitative rather than metric-based, the reports make it possible to examine not only the final annotations, but also the intermediate decisions that produced them.

```
SEMANTIC TABLE ANNOTATION RUN REPORT
====================================



================================================================================
00 - PIPELINE RUN OVERVIEW
================================================================================
report_name: report_001_225433.txt
report_folder: logs\runs\2026-05-29
run_number_today: 1
timestamp: 225433
dataset_name: custom
dataset_split: experiment
dataset_path: data/experiments/exp3_custom/worlds_best_restaurants.csv
llm_provider: groq
llm_model: openai/gpt-oss-20b
kg_source: dbpedia
run_until: cta
```

Figure 4-6: Report log run example

4.7 CLI

Command-line execution was added to make the pipeline easier to run without changing the source code for every experiment. This part is implemented in ***src/pipeline/cli.py***, which acts as a simple interface between the user and the main pipeline. Instead of modifying configuration values manually inside the code, the user can provide important execution parameters directly from the terminal.

The CLI supports execution on a single CSV file or on a folder containing multiple CSV files. In the single-file case, the pipeline runs once and generates one report. In the folder case, the system runs the pipeline separately for each CSV file, producing separate reports for each table. This is useful for testing multiple custom tables or selected benchmark examples in a consistent way.

The CLI also allows the user to select important experimental options, such as the LLM provider, model name, knowledge graph source, stopping point of the pipeline, number of representative values, CEA row limit, and routing threshold. This makes it possible to run quick partial tests, such as stopping after routing, or full annotation tests without editing the implementation.

A typical full run command is shown below:

```
python -m src.pipeline.cli
--file data/experiments/exp2_tt/tt_2.csv
--dataset-name toughtables
--split experiment
--provider groq
--llm "openai/gpt-oss-20b"
--kg dbpedia
--run-until cta
--sample-values 5
--kg-candidate-limit 3
--cea-max-rows 5
--cea-max-cells 8
--cta-max-entities 10
--cta-types-per-entity 5
--router-table-weak-ratio 0.70
--reuse-threshold 0.20
--ner true
--value-enrichment true
--cea-reuse true
```

Figure 4-7: CLI example commands

This execution interface supports the experimental workflow of the project. It allows different tables, providers, models, and parameters to be tested in a reproducible way, while the generated reports keep a separate record of each run.

5. Workflow Scenarios

The previous section described the implementation components separately. This section explains how these components are combined depending on the table scenario selected by the routing module. The goal is to show how the pipeline changes its behavior according to the semantic strength of headers and cell values. The scenario selection is mainly handled by ***src/routing/router.py***, while ***src/pipeline/workflows.py*** organizes the execution of the appropriate tools. The workflow-specific operations are implemented in ***src/tools/topic_detection.py***, ***src/tools/cea_tools.py***, and ***src/tools/cta_tools.py***. These modules allow the system to apply different combinations of topic detection, CEA, and CTA depending on the semantic strength of headers and cell values.

5.1 Weak Headers + Strong Cells

This scenario occurs when the column name is not semantically useful, but the cell values contain enough meaningful information to understand the column. In this case, the header alone cannot support CTA, but the cell values can still provide useful semantic evidence. Therefore, the workflow first applies topic detection in order to infer a more meaningful working column name. After that, the system can continue with CEA and CTA, using knowledge graph candidates and LLM-based context-supported selection.

For this scenario, the workflow can be shown as:

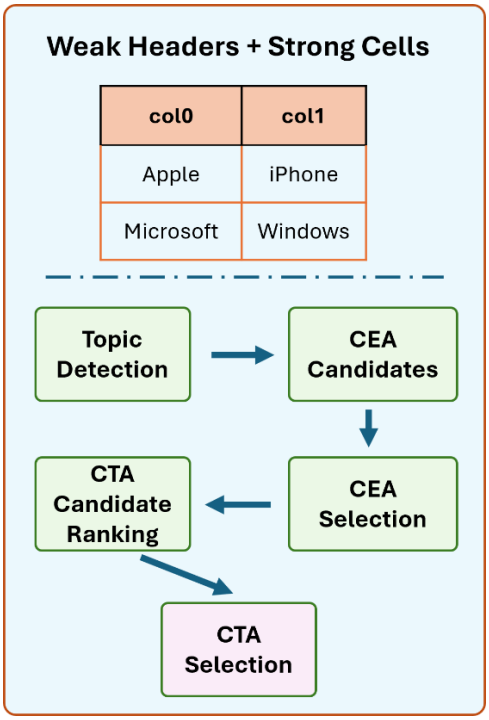

Figure 5-1: Weak Headers - Strong Cells Scenario

5.2 Strong Headers + Weak Cells

This scenario happens when the column name is meaningful, but the cell values are not suitable for entity linking. The cells may be numeric, date-like, identifier-like, coded, abbreviated, or too generic. This scenario is useful for scientific or measurement-heavy tables, where many columns contain the aforementioned cell types. In this case, knowledge graph lookup for individual cells may return poor or misleading candidates.

For this reason, CEA is skipped. The system does not try to force entity links for values that do not represent clear entities. Instead, CTA is performed using the column name and the broader table context as the column name and other headers become the main contextual evidence for CTA.

For this scenario, the workflow can be shown as:

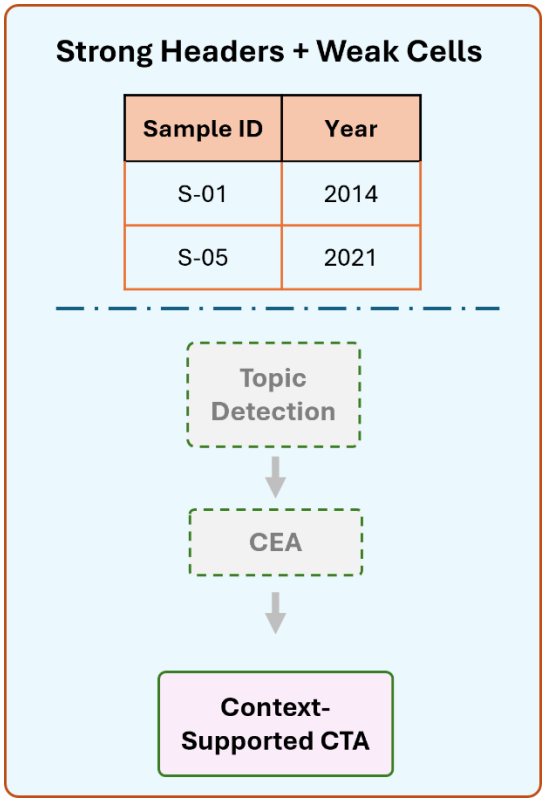

Figure 5-2: Strong Headers - Weak Cells Scenario

In the implementation, weak-cell cases are prioritized during routing. This means that when the cells of a column are mostly numeric, identifier-like, empty, or otherwise unsuitable for entity linking, the column is routed to the weak-cell workflow even if the header itself is also weak. The header weakness is still recorded, but no separate weak-header/weak-cell workflow is used. Instead, the system skips CEA and relies on the available header and table context for CTA. This

avoids forcing knowledge graph annotations on values that are unlikely to represent meaningful entities.

5.3 Strong Headers + Strong Cells

This scenario occurs when both the column header and the cell values provide useful semantic information. It is the most direct annotation case, because the header already gives a meaningful description and the cells can also be linked to knowledge graph entities.

In this case, topic detection is not needed because the column name is already informative. The system proceeds directly to CEA candidate generation and selection. The selected cell entities can then support CTA by providing ontology classes or semantic types related to the column.

For this scenario, the workflow can be shown as:

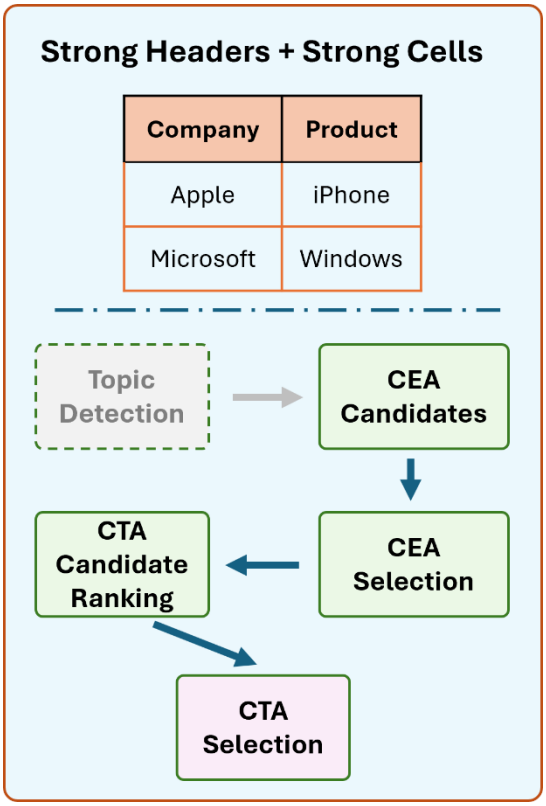

Figure 5-3: Strong Headers - Strong Cells Scenario

6. Experiments & Results

This section presents the experimental evaluation of the implemented semantic table annotation pipeline. The goal of the experiments is not to reproduce the full official benchmark evaluation of the original paper, but to examine how the implemented system behaves under different table conditions, datasets, model providers, and other parameter configurations. The experiments focus on the main stages of the pipeline, including preprocessing, representative value extraction, routing, topic detection, CEA candidate generation and selection, CTA ranking and selection, and report generation. The experimentation step follows a visual comparison between our results and the selected ground truth datasets instead of metric-supported evaluation to distinguish the results in a more sophisticated way.

6.1 Experimental Setup

The experiments were conducted using a combination of benchmark-style tables and small **Kaggle** datasets. The selected benchmark cases were taken from **BiodivTab** and **ToughTables**, since these datasets are also used in the selected paper for evaluating CTA and CEA under difficult table annotation conditions. The Kaggle datasets were used as additional custom examples in order to test the pipeline on clearer business, travel, and brand-related tables. For some custom datasets, the original headers were manually replaced with generic names such as *col0*, *col1* etc. in some datasets, in order to simulate the weak-header scenario and test the topic detection stage.

Four **LLM** configurations were tested: *llama-3.1-8b-instant, llama-3.3-70b-versatile, gemini-3.1-flash-lite, and openai/gpt-oss-20b*, although the final two were used in the final experiments. These models were selected to compare a strong free API model, fast lightweight Groq models, a stronger open-model API option, and a local Ollama model. Local Ollama models were considered, but were not included in the final runs due to hardware and runtime constraints. To keep the comparison manageable, each experiment was executed on small table slices, usually between 10 and 20 rows for CEA and 3 to 5 selected columns for CTA annotations. For **Knowledge Graphs,** we use the DBpedia and the Wikidata Client. The prompt templates are already established in ***prompts.yaml*** and each experiment produced a report in the ***runs*** folder of ***logs***.

The **main parameters** controlled during the experiments were the number of representative values, the number of KG candidates, CEA row/cell limits, CTA candidate limits, routing threshold, the use of value enrichment, and the use of CEA reuse. The baseline configuration used routing, NER, KG lookup, CEA reuse, and compact TXT reporting, while value enrichment was tested separately because it increases the number of LLM calls.

In summary, the following experiments were conducted:

Table 6-1: Experiment parameters summary

Dataset(s)	LLM/KG	Scenario	Parameters
ToughTables / tt_2.csv	Groq / openai/gpt-oss-20b /DBpedia	WH+SC	sample-values 5 cea-max-cells 8 router-table-weak-ratio 0.70 cta-max-entities 10 value-enrichment true
BiodivTab / bdt_4.csv	Groq / openai/gpt-oss-20b /Wikidata	SH+WC	sample-values 5 cea-max-cells 10 router-table-weak-ratio 0.70 cta-max-entities 10 value-enrichment true
Kaggle / worlds_best_restaurants.csv	Groq / openai/gpt-oss-20b /DBpedia	SH+SC	sample-values 5 cea-max-cells 10 router-table-weak-ratio 0.70 cta-max-entities 10 value-enrichment true
Kaggle / top_ecommerce_brands.csv	Google / gemini-3.1-flash-lite /DBpedia	WH+SC	sample-values 5 cea-max-cells 8 router-table-weak-ratio 0.70 cta-max-entities 10 value-enrichment true

6.2 Benchmarks

The benchmark experiments are used to demonstrate how the pipeline behaves on more difficult table annotation cases. From BiodivTab and ToughTables, a small number of representative tables were selected according to the expected workflow scenario. The selected cases were not intended to provide a complete benchmark score, but to show whether the routing decisions, CEA outputs, CTA outputs, and generated reports are consistent with the type of table being processed.

6.2.1 BiodivTab

The **BiodivTab** examples were mainly used to test cases where column headers are more informative, but some cell values may be abbreviations, codes, numeric values, or domain-specific biological terms. These tables are useful for testing whether the router can identify weak-cell columns and whether CTA can still be performed using header and table context.

On the experiment we did on one dataset, the router classified the table as a strong-header/weak-cell-heavy case. As a result, topic detection and CEA were skipped. This was the

expected behavior because most cells did not represent clear knowledge graph entities. Instead, CTA was performed using the available headers and table context.

The final CTA outputs were meaningful for the inspected columns. For example, *Samplenr* was typed as *Sample ID*, *Seedlingnr* as *Seedling ID*, *Year* as *Date*, *Height_P* as *Plant Height*, *Leaves_Liv* as *Living leaf count*, and *Biomass_Above* as *AboveGroundBiomass*.

The results, along with a small table split for showcase, are summarized below:

Table 6-2: BiodivTab examined table

Planted_Species	Treatment	Leaves_Liv	Biomass_Above
D.glaucifolia	Light	23.0	138.04
C.glauca	Light	10.0	-
L.glaber	Shadow	0.0	-
D.glaucifolia	Light	-	0.66
Q.serrata	Shadow	-	1.29

Table 6-3: BiodivTab result comparison

Evidence Inspected	Expected Semantic Meaning	Pipeline Output	Judgement
Cell: L.glaber	Plant Species	**Value Enrichment**: Liriodendron glaber	**Close**
Cell: Q.serrata	Plant Species	**Value Enrichment**: Quercus serrata	**Close**
Header: Year	**Cell value**: 2015 (July)	**CTA(LLM)**: Date	**Close**
Header: Samplenr	-	**CTA(LLM)**: Sample ID	**Close**

Header: Height_P	-	**CTA(LLM)**: Plant Height	**Close**
Header: Biomass_Above	-	**CTA (LLM)**: AboveGroundBiomass	**Close**
Header: Planted_Species	**KG**: species	**CTA (LLM)**: Plant Species	**Close**
Header: Density	**KG**: mass density	**CTA (LLM)**: Density	**Close**

6.2.2 ToughTables

The **ToughTables** examples were mainly used to test more difficult annotation conditions, especially ambiguous entities, noisy values, spelling issues, and weak or missing headers. These cases are useful for evaluating topic detection, KG candidate generation, and context-supported CEA selection. This experiment focuses on whether the system can infer meaningful working column names, retrieve relevant KG candidates, and use row context to select appropriate entities.

As a result, the router classified the table as a weak-header/strong-cell case for most columns. This triggered topic detection, followed by CEA and CTA. Topic detection produced meaningful working column names, such as *Person Name, Date, City, U.S. state*, and *Country*.

The CEA stage produced KG-supported annotations for clear entity values. For example, *Ben Chapman* was linked to *Ben Chapman (baseball), Nashville* was linked to *Nashville, Tennessee* and *United States* was linked to *United States*. The CTA stage produced final types such as *Person*, *Date, City, AdministrativeRegion,* and *Country*.

Table 6-4: ToughTables examined table

col0	col1	col2	col3	col4
Ben Chapman	1908-12-25	Nashville	Tennessee	United States
John Noel	1888-02-06	Nashville	Tennessee	United States
William Schneider	1959-04-25	Syracuse	New York	United States

Kevin Lee	1971-01-01	Mobile	Alabama	United States
Angela Little	1972-07-22	Albertville	Alabama	United States

Table 6-5: ToughTables result comparison

Evidence Inspected	Expected Semantic Meaning	Pipeline Output	Judgement
Cell: Ben Chapman	**KG**: Ben_Chapman_(baseball)	**NER**: PERSON **CEA (KG)**: Ben Chapman (baseball)	**Close**
Cell: John Noel	**KG**: John_Noel_(sport_shooter)	**CEA (KG)**: John Noel	**Close**
Header: col0	**KG**: Agent	**Topic**: Person Name **CTA (KG)**: Person	**Partial**
Header: col2	**KG**: Settlement	**CTA (KG)**: City	**Partial**
Header: col3	**KG**: AdministrativeRegion	**Topic**: U.S. state **CTA (KG)**: AdministrativeRegion	**Close**

6.3 Kaggle / Custom

The **Kaggle** experiments were used to test the implemented pipeline on smaller and more understandable real-world tables so we can see how the pipeline works in a real-world scenario without fixed targets, like the benchmarks. The selected datasets include brand ranking data, top e-commerce brands, world's best restaurants, and worldwide travel cities. These datasets contain recognizable entities such as brands, companies, restaurants, cities, countries, and sectors, making them suitable for qualitative evaluation of CEA and CTA. One dataset was also

modified by replacing its original headers with generic column names in order to test the weak-header workflow.

The **restaurant dataset** was used as a clearer real-world example with strong headers and mostly meaningful entity-like values. The inspected columns included *restaurant*, *location*, *country*, *lat*, and *lng*. The router identified semantic columns such as *restaurant*, *location*, and *country* as strong-cell columns, while numeric columns such as *year*, *rank*, *lat*, and *lng* were treated as weak-cell columns.

Since the headers were already meaningful, topic detection was skipped. CEA was applied to entity-like columns. The pipeline selected correct KG annotations for values such as *El Bulli*, *London*, and *Spain*. One interesting case was *Roses*, where the top KG candidate was noisy (*Guns N' Roses*). In this case, the LLM fallback avoided selecting the misleading candidate and returned a more general location-based annotation.

CTA produced reasonable final types: restaurant was typed as Restaurant, location as City, country as Country, lat as Latitude, and lng as Longitude.

Table 6-6: Kaggle examined table (1)

year	restaurant	location	country
2002	Rockpool	Sydney	Australia
2003	Tangerine	Philadelphia	United States
2005	Vong	New York	United States
2005	Felix	Hong Kong	Hong Kong
2006	Chez Panisse	Berkeley	United States

Table 6-7: Kaggle result comparison (1)

Evidence Inspected	Expected Semantic Meaning	Pipeline Output	Judgement
Cell: Spain	Spain	**CEA (KG)**: Spain	**Close**

Cell: London	London	**CEA (KG)**: London	**Close**
Header: location	location	**CTA (KG)**: City	**Close**
Header: country	country	**CTA (KG)**: Country	**Close**
Header: lat, lng	latitude, longitude	**CTA (LLM)**: Latitude, Longitude	**Close**

This experiment shows the value of combining KG candidates with LLM-based contextual selection. KG lookup is useful when candidates are correct, but LLM selection helps prevent obviously misleading candidates from being accepted blindly.

The second custom dataset we experimented on was the **e-commerce dataset** which was used to test weak-header behavior on a more understandable custom table. The original semantic columns were represented with generic headers such as *col_0, col_1*, *col_2*, and *col_3*, while the values represented *companies*, *founding years*, *founders*, and *countries*.

The router classified most columns as weak-header/strong-cell columns. Topic detection recovered meaningful column names: *col_0* became *E-commerce Companies*, *col_1* became *Year*, *col_2* became *Business Founders*, and *col_3* became *Country*.

CEA selected strong KG annotations for recognizable values. For example, *Amazon* was linked to *Amazon (company)*, *Jeff Bezos* to *Jeff Bezos*, *USA* to *United States*, *Alibaba* to *Alibaba Group*, and *Jack Ma* to *Jack Ma*. CTA then produced clean column types: *Company*, *Year*, *Person*, and *Country*.

Table 6-8: Kaggle examined table (2)

col_0	col_1	col_2	col_3
Amazon	1994	Jeff Bezos	USA
Alibaba	1999	Jack Ma	China
Shopify	2006	Tobias Lütke	Canada

PDD Holdings	2015	Colin Huang	China
eBay	1995	Pierre Omidyar	USA

Table 6-9: Kaggle result comparison (2)

Evidence Inspected	Expected Semantic Meaning	Pipeline Output	Judgement
Cell: Amazon	Amazon	**CEA (KG)**: Amazon (company)	**Close**
Cell: USA	USA	**CEA (KG)**: United States	**Close**
Cell: Alibaba	Alibaba	**CEA (KG)**: Alibaba Group	**Close**
Header: col_0	brand_name	**Topic Detection:** E-commerce Companies **CTA (KG)**: Company	**Close**
Header: col_2	founder	**Topic Detection:** Business Founders **CTA (KG)**: Person	**Partial**
Header: col_3	country	**CTA (KG)**: Country	**Close**

6.4 Results / Findings

Across the selected experiments, the pipeline behaved consistently with the intended workflow design. Weak-header tables benefited from topic detection, which provided useful working names before CEA and CTA, while in strong-header/weak-cell tables, CEA was skipped, and CTA relied on header and table context. Strong-header/strong-cell tables skipped topic detection and proceeded directly to KG-supported annotation stages.

The generated reports were useful for understanding the behavior of the system. They showed not only the final annotations, but also the table scenario, column scenarios, representative values, NER hints, value enrichment behavior, KG candidates, selected CEA annotations, CTA

outputs, and skipped stages. This made it possible to inspect whether the pipeline followed the expected path for each dataset.

Most selected outputs were semantically close to the expected meaning. Partial cases appeared mainly when the predicted type was broader or narrower than the expected ground-truth type, or when KG lookup returned noisy candidates. The limits of the experiments were the small LLM call-capacity and the limits of the free-tier API models available to us. Furthermore, the above limitation also hindered the experiment with the Google LLM, where we had to disable the value enrichment module, as well as the small number of CEA rows and CTA candidates. However, the combination of KG candidates and LLM-based selection generally produced understandable and useful annotations for the inspected examples.

7. Modifications & Extensions

This section summarizes the main adaptations and extensions introduced in the implemented project. The selected paper describes the overall **ReAct-based STA** method, its tools, and its experimental results, but the full implementation code is not available for direct comparison. Therefore, the differences discussed here refer to the implemented project in relation to the paper's described methodology.

7.1 Differences from the Paper

The main structural difference is that the **workflow selection** in this project is implemented in a more deterministic way. The paper presents the method as a ReAct-based agent that dynamically selects suitable tool combinations according to the characteristics of the table. In this implementation, the same scenario-based idea is operationalized through an explicit routing module and workflow layer. The router identifies whether headers and cell values are semantically strong or weak, and the pipeline then executes or skips the corresponding tools, making the execution easier to inspect, debug, and reproduce.

Another difference is the **experimental scope**. The paper evaluates its approach mainly on Tough Tables and BiodivTab, which are SemTab benchmark datasets. In this project, the system is designed to run not only on selected benchmark-style examples, but also on custom CSV tables and datasets from different domains. This makes the implementation more suitable for demonstration and qualitative inspection across different table structures.

The project also introduces a broader fallback behavior for annotation decisions. When a knowledge graph entity or type candidate is suitable, the system can use KG-grounded annotation. However, when the value does not correspond clearly to a knowledge graph entity, the pipeline can rely on LLM-based semantic annotation using available table context.

Additionally, the **value enrichment module** is used before annotation to clarify or normalize representative values when needed. Its role is to improve the semantic evidence that later components receive. This is useful when table values contain abbreviations, noisy forms, or unclear textual representations. Similarly, the paper uses LLM to inspect cells in context, repair misspellings, and expand abbreviated forms.

7.2 Practical Improvements

The implementation supports multiple knowledge graph sources. The paper uses DBpedia as the main ontology-based knowledge graph, while this project structures the KG layer so that different

sources can be selected or extended. This allows experiments with **DBpedia**, **Wikidata**-style lookup, or future knowledge graph backends without changing the main pipeline design.

Another important extension is support for multiple LLM execution modes. The system can use API-based providers, but it can also run local models through Ollama. This makes the project more flexible in terms of cost, privacy, and experimentation. API models can provide stronger annotation quality, while local models can be useful for free testing, debugging, or offline experimentation.

The project also includes a command-line interface and automatic reporting. The CLI allows the pipeline to run on one CSV file or on a folder of input files, while the reporting module generates a compact *.txt* report for each run.

The pipeline is also practical for experimentation across datasets. Instead of requiring fixed benchmark input only, the CLI and folder execution make it possible to test custom tables, selected benchmark examples, and different domain datasets. This helps demonstrate how the pipeline behaves under different table conditions, such as weak headers, weak cells, or clean semantic columns.

8. Conclusion

This project implemented a modular Semantic Table Annotation pipeline inspired by the paper *An LLM Agent-Based Complex Semantic Table Annotation* Approach. The system focuses on the two main annotation tasks of the paper, Cell Entity Annotation and Column Type Annotation, and combines preprocessing, routing, knowledge graph lookup, LLM-based selection, and compact reporting in a single executable pipeline.

The implementation follows the paper's scenario-based annotation idea, where different table conditions require different workflows. Weak headers, weak cells, and strong semantic columns are handled through an explicit routing mechanism that decides which tools should be executed or skipped. This makes the system adaptive, while also keeping the execution transparent and reproducible.

The experiments showed that the implemented routing logic behaved as expected across the selected examples. In the ToughTables experiment, weak generic headers were recovered through topic detection and then used to support CEA and CTA. In the BiodivTab experiment, the router identified a weak-cell-heavy table, skipped CEA, and produced useful CTA labels from the available headers and table context. In the custom/Kaggle experiments, the pipeline produced mostly close annotations for real-world entities such as restaurants, locations, countries, companies, founders, and years. These results support the idea that combining KG-grounded candidates with LLM-based contextual decisions can produce useful and inspectable semantic annotations.

Overall, the project demonstrates how an LLM-agent-based semantic annotation method can be transformed into a practical and modular software prototype. The implementation does not reproduce the full official benchmark evaluation of the paper, and the final annotation quality still depends on the selected LLM provider, knowledge graph coverage, and the quality of the input tables. As another limitation, the source code of the paper not being public, hides vital details that could clarify how certain modules behave. A final limitation is that exact comparison with ground truth is not always straightforward. Some datasets are more compatible with DBpedia-style annotations, while others involve Wikidata-style or domain-specific semantics. In addition, LLM outputs can vary across providers and models, so the generated reports are important for inspecting the behavior of each run rather than relying only on final labels. However, the system provides a clear and extensible foundation for further experimentation and future improvements.

References

1. Bizer, Christian, Jens Lehmann, Georgi Kobilarov, et al. “DBpedia - A Crystallization Point for the Web of Data.” *Journal of Web Semantics*, The Web of Data, vol. 7, no. 3 (2009): 154–65. https://doi.org/10.1016/j.websem.2009.07.002.

2. Geng, Yilin, Shujing Wang, Chuan Wang, et al. “An LLM Agent-Based Complex Semantic Table Annotation Approach.” In *Advanced Data Mining and Applications*, edited by Masatoshi Yoshikawa, Xiaofeng Meng, Yang Cao, Chuan Xiao, Weitong Chen, and Yanda Wang. Springer Nature, 2026. https://doi.org/10.1007/978-981-95-3456-2_16.

3. Liu, Jixiong, Yoan Chabot, Raphaël Troncy, Viet-Phi Huynh, Thomas Labbé, and Pierre Monnin. “From Tabular Data to Knowledge Graphs: A Survey of Semantic Table Interpretation Tasks and Methods.” *Journal of Web Semantics* 76 (April 2023): 100761. https://doi.org/10.1016/j.websem.2022.100761.

4. “Semantic Web Challenge on Tabular Data to Knowledge Graph Matching.” Accessed May 15, 2026. https://sem-tab-challenge.github.io/2022/.

5. Yao, Shunyu, Jeffrey Zhao, Dian Yu, et al. “ReAct: Synergizing Reasoning and Acting in Language Models.” arXiv:2210.03629. Preprint, arXiv, March 10, 2023. https://doi.org/10.48550/arXiv.2210.03629.